ASSISTED BY AI
https://ai-label.org/

Botones y campos de texto

Componente	Cuándo usarlo	Listener y registro	Métodos del listener	Particularidades del uso
JButton	Ejecutar una acción al pulsar el botón.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	La activación puede hacerse con ratón o teclado. La referencia al botón se obtiene con (JButton) e.getSource().
JTextField	Confirmar el texto con Enter.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	El texto se obtiene con textField.getText().
JTextField	Detectar inserciones y borrados, incluido pegar texto.	DocumentListener getDocument(). addDocumentListener(...)	insertUpdate(DocumentEvent e) removeUpdate(DocumentEvent e) changedUpdate(DocumentEvent e)	El texto se obtiene con textField.getText(). Implementa los tres métodos. changedUpdate detecta cambios de atributos o estilo. Este listener se activa cuando el texto ya ha cambiado. No uses no uses setText(...) dentro del listener. Si quieres cambiar el texto dentro del listener usa DocumentFilter.
JPasswordField	Confirmar la contraseña con Enter.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	La contraseña se obtiene con passwordField.getPassword(): devuelve char[].
JPasswordField	Detectar cambios de contraseña.	DocumentListener getDocument(). addDocumentListener(...)	insertUpdate(DocumentEvent e) removeUpdate(DocumentEvent e) changedUpdate(DocumentEvent e)	La contraseña se obtiene con passwordField.getPassword(). El comportamiento es similar al de JTextField.
JList	Reaccionar al cambio de selección.	ListSelectionListener addListSelectionListener(...)	valueChanged(ListSelectionEvent e)	Si no hay selección getSelectedIndex() devuelve -1 y getSelectedValue() devuelve null. Para selección múltiple, usa getSelectedIndices() o getSelectedValuesList(). El listener detecta la interacción completa desde que se hace el primer click para cambiar el valor seleccionado hasta que se confirma la selección. Si solo interesa el estado final, empieza con if (e.getValueIsAdjusting()) return.

Componentes de opciones (Combo/Check) y tablas

Componente	Cuándo usarlo	Listener y registro	Métodos del listener	Particularidades del uso
JComboBox	Reaccionar a una selección o la confirmación de edición.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	comboBox.getSelectedItem(); puede ser null. En un combo editable, el valor puede no estar en la lista. setSelectedItem(...) también puede disparar el evento: no siempre lo provoca una interacción directa con el componente.
JComboBox	Distinguir selección y deselección de elementos.	ItemListener addItemListener(...)	itemStateChanged(ItemEvent e)	e.getItem() es el elemento afectado. Un cambio puede generar DESELECTED para el anterior y SELECTED para el nuevo. Si solo interesa el nuevo, usa if (e.getStateChange() == ItemEvent.SELECTED) { ... }.
JCheckBox	Ejecutar una acción al activar la casilla.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	Lee checkBox.isSelected(). Un cambio mediante setSelected(...) no activa el listener
JCheckBox	Detectar cambios del estado seleccionado.	ItemListener addItemListener(...)	itemStateChanged(ItemEvent e)	Compara e.getStateChange() con ItemEvent.SELECTED o DESELECTED. Para obtener la casilla, usa (JCheckBox) e.getSource(); e.getItem() también representa la casilla.
JRadioButton	Ejecutar una acción al activar una opción.	ActionListener addActionListener(...)	actionPerformed(ActionEvent e)	Usa ButtonGroup si las opciones deben ser excluyentes. Identifica el botón por referencia o asigna setActionCommand(...) y lee e.getActionCommand().
JRadioButton	Detectar la selección o deselección de una opción.	ItemListener addItemListener(...)	itemStateChanged(ItemEvent e)	Al cambiar la opción de un grupo, se deselecciona una y se selecciona otra. Filtra SELECTED si solo quieres actuar sobre la nueva. e.getSource() y e.getItem() identifican la opción afectada.
JTable	Reaccionar a cambios de las filas o columnas seleccionadas.	ListSelectionListener getSelectionModel(). addListSelectionListener(...)	valueChanged(ListSelectionEvent e)	Usa if (e.getValueIsAdjusting()) return;. table.getSelectedRow() devuelve -1 si no hay selección. e.getSource() es el modelo de selección, no la tabla. **Para seleccionar columnas.** Registra el ListSelectionListener en table.getColumnModel().getSelectionModel().
JTable	Reaccionar a cambios en los datos o estructura del modelo.	TableModelListener getModel(). addTableModelListener(...)	tableChanged(TableModelEvent e)	Consulta getType() (INSERT, UPDATE o DELETE), getFirstRow(), getLastRow() y getColumn(). Puede afectar a varias filas, a ALL_COLUMNS o a la estructura (HEADER_ROW). Los índices pertenecen al modelo.

Ratón, teclado y foco

Componente	Cuándo usarlo	Listener y registro	Métodos del listener	Claves de uso
Teclado	Detectar teclas, caracteres o combinaciones.	KeyListener / KeyAdapter addKeyListener(...)	keyPressed(KeyEvent e) keyReleased(KeyEvent e) keyTyped(KeyEvent e)	El componente debe admitir foco y tenerlo. En pressed/released usa getKeyCode(); en typed, getKeyChar(). Para Ctrl+E: e.isControlDown() && e.getKeyCode() == KeyEvent.VK_E, dentro de keyPressed. KeyAdapter permite sobrescribir solo los métodos necesarios. Usa KeyListener para eventos de teclado sobre el componente que tiene el foco. Para saber si cambia el texto de un campo, usa DocumentListener.
Ratón	Detectar clics y entrada o salida de un componente.	MouseListener / MouseAdapter addMouseListener(...)	mouseClicked(MouseEvent e) mousePressed(MouseEvent e) mouseReleased(MouseEvent e) mouseEntered(MouseEvent e) mouseExited(MouseEvent e)	En clic/pulsación/liberación, getButton() identifica el botón. En mouseClicked, getClickCount() == 2 detecta doble clic. getX() y getY() son coordenadas relativas al componente. mouseEntered y mouseExited detectan que el puntero entra o sale del componente
Movimiento	Seguir el puntero o un arrastre.	MouseMotionListener / MouseMotionAdapter addMouseMotionListener(...)	mouseMoved(MouseEvent e) mouseDragged(MouseEvent e)	En JTable usa rowAtPoint(e.getPoint()) y columnAtPoint(e.getPoint()); comprueba -1. Para un arrastre completo, puedes combinarlo con pressed/released. MouseAdapter también admite movimiento, pero debes registrarlo con addMouseMotionListener.
Rueda	Implementar zoom o desplazamiento personalizado.	MouseWheelListener addMouseWheelListener(...)	mouseWheelMoved(MouseWheelEvent e)	getWheelRotation() indica pasos y dirección; getPreciseWheelRotation() permite rotaciones fraccionarias.
Foco (cursor o selección activa)	Reaccionar al ganar o perder el foco.	FocusListener / FocusAdapter addFocusListener(...)	focusGained(FocusEvent e) focusLost(FocusEvent e)	isTemporary() distingue cambios temporales. Puedes comprobar datos al perder foco

Cómo implementar listeners

En Java Swing, la gestión de eventos se basa en el patrón *Observer/Listener*, donde los *listeners* se definen mediante interfaces. Dado que en Java no se pueden instanciar interfaces de forma directa, existen varias alternativas para definir y registrar la respuesta a un evento. A continuación, se muestran 3 alternativas tomando como referencia un evento de click de un botón (JButton)

Clase anónima guardada en una variable

Consiste en instanciar la interfaz creando una clase anónima y sobreescribiendo sus métodos.

- **Ventajas**: Permite separar la creación del *listener* del momento en que se añade al componente.
- **Reutilización**: Facilita la asignación del mismo comportamiento a varios componentes.

```java
// Creación de la instancia mediante una clase anónima guardada en variable
ActionListener miListener = new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("¡Botón pulsado!");
    }
};

// Registro del listener en varios componentes (Reutilización)
miBoton1.addActionListener(miListener);
miBoton2.addActionListener(miListener);
```

Clase anónima como parámetro del método addListener

La clase anónima se define e instancia como argumento dentro de la llamada al método de registro del listener (addActionListener, addMouseListener, etc.).

- **Ventaja**: Mantiene la lógica del evento agrupada con en el componente que lo dispara.
- **Limitación**: El comportamiento no se puede reutilizar entre componentes. Hay que duplicar código.

```java
// Definición directa de la clase anónima dentro de addActionListener
miBoton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("¡Botón pulsado!");
    }
});
```

Expresiones Lambda

Una expresión lambda en Java es una función anónima (un método sin nombre) que permite definir un comportamiento de forma breve y compacta. Permiten pasar "código que hace algo" como si fuera un parámetro.

- **Requisito imprescindible**: Las funciones lambdas solo pueden utilizarse con interfaces que tengan un único método, como por ejemplo ActionListener (con actionPerformed).
- **Limitaciones**:
  - El comportamiento no se puede reutilizar entre componentes. Hay que duplicar código.
  - No se puede utilizar con listeners que tienen más de un método como DocumentListener o MouseListener. En esos casos hay que usar clases anónimas o adaptadoras como MouseAdapter.

```java
// Expresión lambda
miBoton.addActionListener(e -> {
    System.out.println("¡Botón pulsado!");
});

// Versión simplificada si solo se ejecuta una sentencia de código
miBoton.addActionListener(e -> System.out.println("¡Botón pulsado!"));
```